HYPERION-20: A Reference Architecture for Orchestrating Twenty Classes of Computing Processors

A Conceptual Systems Research Paper and Experimental Evaluation Proposal

1 November 2026

Abstract

This paper proposes HYPERION-20, a research architecture for coordinating twenty named categories of processing units within a logically unified heterogeneous computing environment. The proposed platform joins conventional central and graphics processors with tensor and language accelerators, programmable logic, infrastructure offload engines, wafer-scale computers, neuromorphic systems, photonic accelerators, quantum processors, thermodynamic sampling units, and experimental biological neural interfaces. Its principal contribution is not a claim that all twenty technologies can share one memory space or that their combination automatically improves computational performance. Instead, the paper specifies a capability-aware runtime, explicit data-movement interfaces, safety and governance boundaries, and a multi-objective scheduler that selects execution resources according to program compatibility, predicted latency, data locality, energy, uncertainty, and reliability. A staged deployment model separates commercially deployable components from remotely accessed or experimental research systems. A reproducible evaluation plan is offered for assessing end-to-end latency, throughput, energy per accepted result, performance portability, scheduling regret, robustness, and scientific validity. As of November 2026, HYPERION-20 remains a proposed architecture: no integrated installation, hardware measurements, speedups, or energy savings are claimed. The paper argues that the strongest research opportunity lies in making genuinely different computational models interoperable without erasing their semantics or overestimating their maturity.

1. Introduction

Contemporary high-performance computing no longer scales solely by increasing the clock frequency of a general-purpose processor. Systems now combine CPUs with graphics accelerators, domain-specific matrix engines, programmable logic, specialised networking and storage processors, and increasingly elaborate memory hierarchies. Alternative computational substrates—spiking neural circuits, optical interference networks, programmable stochastic devices, quantum circuits and living neuronal cultures—extend the range of operations that hardware may directly implement. However, these substrates differ not merely in performance but in their fundamental execution semantics, interfaces, physical operating conditions and ability to provide reproducible results.

HYPERION-20 is a proposal for a *logically* unified supercomputing platform integrating twenty processor categories through a common resource and workflow layer. It is deliberately not proposed as a monolithic chip, a single uniform instruction-set architecture or an immediately buildable collection of twenty purchasable components. Rather, the system is a federation of computing domains with explicit contracts for executable functions, input and output types, precision, privacy, timing and failure handling.

The research question is: **Can a capability-aware orchestration architecture improve the utility of highly heterogeneous resources while preserving the performance, scientific integrity and safety constraints of each computational model?** This question is distinct from the claim that

more hardware variety necessarily produces a faster supercomputer. A system that includes a poorly matched accelerator may become slower because of serialisation, queueing, compilation, network traffic or experimental calibration overhead.

The intended contributions of this concept paper are fourfold: (1) a taxonomy and feasibility boundary for twenty functional processor classes; (2) a multi-tier hardware and software reference architecture; (3) a schedulable workload model with an explicit multi-objective cost function and fallback rules; and (4) a testable evaluation protocol with falsifiable criteria. No empirical results are reported, and no implementation of HYPERION-20 is asserted to exist.

2. Scope, Definitions and Processor Taxonomy

The labels used in popular descriptions of processors are not necessarily disjoint. A wafer-scale engine is a highly specialised application-specific integrated circuit (ASIC); an accelerated processing unit (APU) may contain both CPU and GPU cores; and a physics processor may be realised as a GPU workload rather than as a discrete physical device. Likewise, *SPU* can refer to different concepts in different product families. This paper adopts explicit functional definitions so that the system can be described consistently.

Table 1. Functional processor taxonomy used by HYPERION-20.

No.	Class	Working definition	Intended HYPERION-20 function
1	CPU	General-purpose central processing unit	Control plane, sequential processing and fallback
2	GPU	Massively parallel graphics/general compute processor	Dense parallel compute, AI and rendering
3	PPU	Physics processing unit or a dedicated physics-execution backend	Collision, particle and rigid-body simulation
4	APU	Integrated CPU–GPU accelerated processing unit	Edge orchestration, display and local execution
5	SPU	Sound or signal processing unit, as defined here	Spatial audio and low-latency signal processing
6	NPU	Neural processing unit	Efficient local neural inference
7	TPU	Tensor processing unit	Systolic-array tensor computation
8	LPU	Language processing unit in a streaming-inference architecture	Low-latency model inference
9	MCU	Microcontroller unit	Power, sensors, interlocks and equipment control
10	ISP	Image signal processor	Camera acquisition and image preprocessing
11	Security processor	Isolated security and key-management execution engine	Root of trust, attestation and cryptography
12	ASIC	Fixed-function application-specific circuit other than separately counted categories	Selected stable kernels and instrument functions
13	FPGA	Field-programmable gate array	Reconfigurable kernels and instrument I/O
14	DPU	Data processing unit	Network, storage and infrastructure offload

No.	Class	Working definition	Intended HYPERION-20 function
15	WSE	Wafer-scale engine	Large on-wafer AI execution
16	Neuromorphic	Spiking/event-driven processor	Sparse sensory streams and SNN research
17	Photonic	Optical computational processor	Supported optical linear algebra and research
18	QPU	Quantum processing unit	Quantum-circuit experiments and hybrid subroutines
19	TSU	Thermodynamic sampling unit	Programmable probabilistic sampling
20	BPU	Biological processing unit, a system-level label used here	Neuronal culture experiments and readout

The taxonomy denotes *service classes*, not twenty independent physical instruction sets. The scheduler must also distinguish whether a backend is production-ready, vendor-dependent, simulated, remotely accessible, or available only under laboratory approval. Not every application requires—or benefits from—using each class. Support for a class is an interoperability target, not a guarantee of universal substitutability.

3. Technical Background and Related Work

3.1 Domain-specific electronic acceleration

Commercial AI computing already provides evidence for specialising the datapath. Google's TPU architecture uses systolic matrix-multiplication units to perform dense tensor operations; the relevant computational advantage arises from mapping compatible matrix workloads onto hardware that minimises unnecessary data movement [1]. Groq's LPU design emphasises software-scheduled streaming execution and on-chip memory for inference, demonstrating a different engineering trade-off from conventional GPU execution [2]. These characteristics are useful precedents, not evidence that an LPU would always outperform a GPU across every model or batch size.

Cerebras introduced its WSE-3 with 900,000 AI cores, 44 GB of on-chip SRAM and a manufacturer-stated 125 PFLOPS peak for applicable AI operations [3]. Those figures describe a specific hardware generation and arithmetic regime. Peak throughput, on-chip memory bandwidth and total-system inference latency must not be equated, and vendor claims must be independently benchmarked for proposed HYPERION workloads. NVIDIA's BlueField and DOCA software illustrate the distinct role of a DPU as an infrastructure processor for networking, storage and security offload [4].

3.2 Alternative physical and computational models

Neuromorphic systems implement event-driven and spiking computation. Intel's Hala Point research platform combined 1,152 Loihi 2 processors and was described as supporting up to 1.15 billion artificial neurons [5]. Neuron count is an architectural capacity measure, not a direct measure of general intelligence or application performance.

Research on photonic computing has demonstrated neural-network workloads on optical accelerator hardware. In 2025, Ahmed and colleagues reported a photonic AI processor capable of running advanced models at near-electronic precision for investigated tasks [6]. Such demonstrations are significant, but input/output conversions, precision, calibration, thermal drift,

packaging and system integration remain material costs. Optical communication and optical computation are distinct functions and should not be conflated.

Quantum computers manipulate quantum states with specialised circuit models and measurement operations. IBM's System Two illustrates the necessity of combining cryogenic QPU infrastructure with conventional classical control and runtime services [7]. General quantum advantage for arbitrary scientific or AI workloads should not be assumed. In a hybrid supercomputer, quantum jobs are specialised experimental stages governed by queueing, shot count, circuit depth, noise and classical pre-/post-processing.

Thermodynamic sampling units provide a different operation: direct samples from programmable distributions, particularly model families compatible with a device's physical stochastic dynamics. Extropic's TSU/XTR-0 research platform shows prototype work at the interface between deterministic hosts and probabilistic circuits [8]. A sample from a TSU is not equivalent to a deterministic tensor operation, and future commercial products must not be treated as already validated deployments.

Finally, Jordan and colleagues documented a remotely accessible research platform using biological neuronal cultures, microelectrode arrays and closed-loop stimulation [9]. This establishes experimental feasibility for interfacing living networks, not equivalence with a reliable, programmable digital processor. The 2026 Nuffield Council on Bioethics report identifies governance questions for neural organoid research that would apply to any biological HYPERION component [10].

3.3 Software interoperability

Kubernetes device plugins demonstrate how resource-specific hardware can be exposed to cluster schedulers without pretending that every accelerator has the same programming model [11]. OpenXLA's PJRT interface similarly offers a practical precedent for integrating distinct executable backends [12]. HYPERION extends these ideas conceptually to services whose outputs may be probability samples, event trains, measured quantum bit strings or time-varying biological responses rather than ordinary tensor buffers.

4. Proposed Architecture

4.1 Physical federation

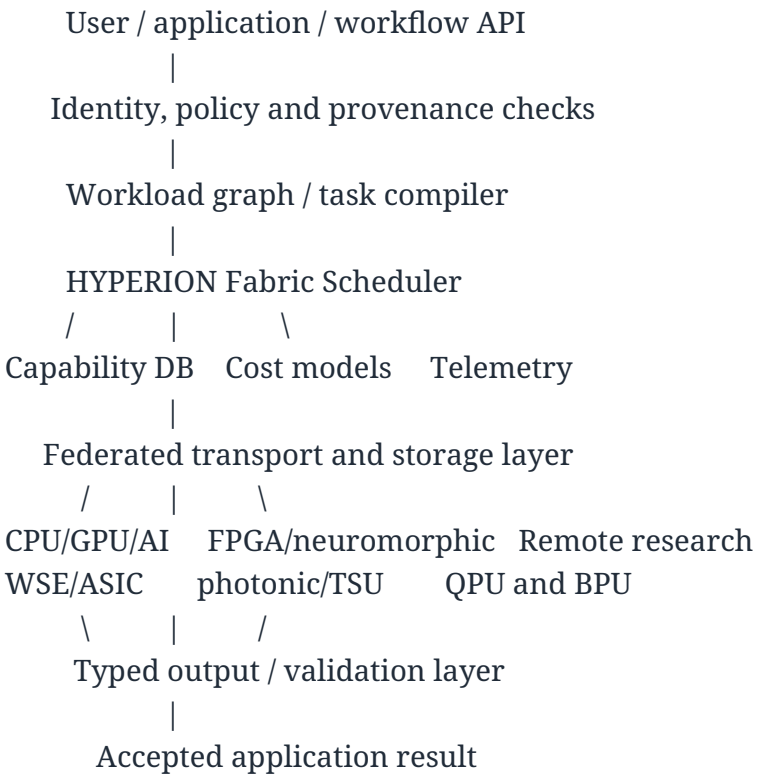
The proposed system consists of six engineering zones under one logical control plane:

1. **Core electronic computing zone:** CPU/GPU/APU nodes, NPU/TPU/LPU acceleration, dedicated ASIC and FPGA equipment, and physics, sound and image-processing services.
2. **Data and trust zone:** DPU-enabled network and storage infrastructure, security processors, identity and key-management services, and immutable audit systems.
3. **Wafer-scale zone:** a separate WSE-compatible appliance with vendor-specific model compilation, cooling and interconnects.
4. **Event-driven and photonic zone:** neuromorphic and optical research appliances with calibration and instrumentation.
5. **Quantum zone:** a cryogenic QPU with dedicated control electronics, preferably accessed as a remote batch service.

6. **Biological zone:** isolated laboratory systems supporting sterile culture maintenance, electrophysiology, stimulation, environmental monitoring and ethics-approved experimentation.

MCUs oversee sensors and fail-safe actuators in relevant zones; they are not substitutes for independent laboratory safety systems. The architecture treats distinct physical domains as separate failure and access-control boundaries. It is neither necessary nor desirable for all components to occupy the same server room.

Figure 1. Logical data and control architecture.



The figure is a logical design. It does not assert a shared processor instruction set, physically shared coherent RAM, or simultaneous availability of all listed experimental devices.

4.2 Five-layer software stack

Layer 1: Physical resources. Devices, host servers, network links, sensors, cooling systems, storage and laboratory instrumentation. Each resource reports firmware, software version, operating conditions, health and supported operations.

Layer 2: Backend adapters. Vendor runtimes, device drivers, compilers, protocol gateways and laboratory control services encapsulate device-specific constraints. Where possible, conventional tensor execution uses existing compiler interfaces; quantum, TSU, neuromorphic and biological backends retain specialised APIs.

Layer 3: Typed intermediate representation. A workflow becomes a directed acyclic graph (DAG) of operations. Every node declares its required semantics: deterministic tensor multiplication, audio filtering, stochastic sample generation, spike processing, quantum circuit execution or biological experiment. Values carry shape, unit, precision, privacy label, uncertainty representation and provenance. A QPU result is a distribution of measured classical outcomes; a biological result is an instrument reading with experimental metadata. Neither is automatically convertible to a tensor without an explicit transformation.

Layer 4: Scheduling and execution. The scheduler discovers feasible backends, estimates total cost, reserves resources, stages data, launches tasks, checks outputs and applies failure-handling policy. Runtime telemetry updates cost estimates and confidence intervals.

Layer 5: Application interfaces. AI training and inference, interactive simulation, scientific optimisation, sensory processing and research experiment management use a common workflow API. Different clients can set different service-level objectives and research constraints.

4.3 Hardware capability descriptors

Each backend publishes a signed manifest including: available operation types; supported data types and precisions; executable model formats; permitted input sizes; memory and storage capacity; host-to-device and network transfer characteristics; compilation costs; isolation capabilities; reservation window; estimated queueing delay; measured power or energy information where available; calibration age; failure modes; reproducibility attributes; and scientific, ethical or legal access restrictions.

For example, a QPU descriptor includes qubit topology, supported gate set, maximum circuit or runtime limits, shots and queue policy. A BPU descriptor includes experiment protocol version, biological source provenance, consent or oversight restrictions where applicable, allowed stimulation envelopes and culture metadata. These fields are operational contracts rather than claims that experimental devices can act as ordinary digital processors.

5. HYPERION Fabric Scheduler

5.1 Workload model

Let a requested computation be a DAG $G=(V,E)$, where V contains tasks and E contains data dependencies. Let D be the set of available backends. A binary decision variable $x_{v,d}$ is one if task v is assigned to device d and zero otherwise. Each task has a declared function signature and a compatibility indicator $a_{v,d} \in \{0,1\}$.

A candidate must first pass *hard* constraints: semantic compatibility, authorised data placement, required precision or scientific validity, device health, capacity, deadline and any safety gate. The scheduler therefore does not route a matrix-multiplication task to a QPU or organoid merely because those resources appear in the inventory.

5.2 Multi-objective optimisation

For a candidate assignment, HYPERION estimates service time T, energy E, money or scarce resource cost K, network/data-movement cost M, and expected invalid-output or failure cost R. One illustrative scalar objective is

$$J=\alpha\,\hat{T}_{end-to-end}+\beta\,\hat{E}_{total}+\gamma\,\hat{K}_{total}+\delta\,\hat{M}_{total}+\epsilon\,\hat{R}_{total}.$$

The weights reflect the application's policy and must operate on normalised, comparably defined quantities. A latency-sensitive assistant may weight T heavily; an energy-limited sensor pipeline may weight E; a scientific experiment may impose strict validity thresholds rather than trading validity against speed. Preferably, the scheduler can expose a Pareto frontier instead of hiding all preferences in a single scalar score.

The assignment constraints include

$$\sum_{d \in D} x_{v,d} = 1, x_{v,d} \le a_{v,d}, \sum_{v \in V} x_{v,d} r_{v,d} \le C_d,$$

where $r_{v,d}$ is reserved resource demand and C_d is usable capacity during the relevant time interval. A production implementation additionally models precedence, preemption, memory limits, device residency, queueing, network contention, thermal constraints and job priorities. The complete optimisation problem is generally combinatorial; an online scheduler therefore needs approximations and periodic replanning.

5.3 Data-aware cost estimation

Cross-device execution costs more than a kernel's device time. A simplified transfer estimate is

$$T_{move}(i, j, b) \simeq L_{ij} + \frac{b}{B_{ij}^{effective}} + T_{serialise} + T_{convert},$$

where L_{ij} is path latency, b is bytes moved, $B_{ij}^{effective}$ is measured effective bandwidth, and the remaining terms capture encoding and representation conversion. For a remote QPU or wet-lab experiment, queueing, preparation, measurement, calibration and policy approval may dominate over network transfer time.

The scheduler should decline an acceleration when

$$T_{stage} + T_{execute,accelerator} + T_{return} \ge T_{execute,baseline}$$

unless another objective, such as energy or accuracy, justifies it. This rule prevents the number of processor categories from being mistaken for a measure of realised performance.

5.4 Scheduling policy and fallback

An initial implementation should use constrained capability filtering followed by prediction-based ranking and conservative admission control. Every job has a designated reference backend—usually CPU or GPU—if one exists. Research workloads without a valid fallback must be labelled as such. Online learning can improve cost predictions, but experimental exploration should occur under user-defined budgets and not silently compromise production service-level agreements.

```
function schedule(task, policy, resource_inventory):
  feasible = []
  for device in resource_inventory:
    if not semantic_match(task, device): continue
    if not permissions_allow(task, device): continue
    if not meets_precision_and_safety(task, device): continue
    if not has_healthy_capacity(task, device): continue
    estimate = predict_end_to_end_cost(task, device)
    feasible.append((device, estimate))
  if feasible is empty:
    return reject_or_reference_fallback(task)
  selected = pareto_rank_and_select(feasible, policy)
  reserve(selected)
  stage_validated_inputs(task, selected)
```

```
result = execute_with_timeout(task, selected)
if not validate_semantics_and_output(result, task):
    return approved_fallback_or_explicit_failure(task)
record_telemetry_and_provenance(task, selected, result)
return result
```

The pseudocode is architectural, not a claim that this scheduler has been implemented or tested.

6. Communication, Memory and Synchronisation

A universal cache-coherent memory space across all twenty classes is not a credible near-term design assumption. CPU and conventional accelerator nodes may use PCIe, supported peer-to-peer transfers, system-specific coherent fabrics and CXL in configurations whose software and hardware actually support them [13]. Networked appliances communicate using RDMA, message passing, RPC or vendor APIs. Some devices expose only bounded command streams or experiment endpoints.

HYPERION instead proposes a **hierarchy of data domains**: local registers and SRAM; device-local high-bandwidth memory; node DRAM; compatible pooled or coherent memory; cluster storage; and remote research endpoints. The compiler and scheduler are responsible for explicit transitions between those domains. Dataset identifiers are content-addressed where practical, and large immutable input objects are cached near expected execution sites. This avoids treating network copies as free.

For deterministic digital kernels, standard numerical error metrics may be appropriate. For stochastic and physical processors, output metadata must be richer. A TSU output includes sample count, parameterisation, estimated sampling bias and mixing diagnostics where applicable; a QPU output includes circuit version, shots, backend calibration data and uncertainty; a BPU output includes recording time, electrode mapping, environmental conditions, biological batch and stimulation protocol. Biological, thermodynamic and quantum outputs are *not* presumed interchangeable just because all may be probabilistic.

7. Security, Reliability, Safety and Scientific Governance

7.1 Trust boundaries

The control plane requires authenticated workloads, least-privilege execution, signed manifests, auditable data transfers and isolation of secrets. Hardware security processors and DPUs may offload selected controls, but software policy remains necessary. Experimental nodes should be isolated from production tenant data unless the transfer has been specifically authorised. Sensitive data must not be routed to a remotely operated research processor solely to optimise latency or cost.

7.2 Fault containment

Failures may include GPU memory errors, FPGA bitstream defects, compiler incompatibilities, network partitions, optical calibration drift, QPU noise excursions, TSU sampling bias and biological culture degradation. Hardware health is not sufficient: task output must be validated against the expected semantics. Deterministic kernels may be retried with checkpointing; stochastic experiments require explicit reproducibility seeds where meaningful, sampling metadata and

statistical checks. Biological experiments cannot be assumed exactly repeatable and may be constrained by protocol duration and sample availability.

A two-tier service model is proposed. **Production tier:** validated CPU/GPU/DPU/NPU/TPU/LPU/WSE/FPGA services under defined service-level objectives. **Research tier:** neuromorphic, photonic, TSU, QPU and BPU components admitted only for compatible approved workloads and with explicit caveats. The tier boundary is based on measured readiness, not on the name of the hardware class; individual devices may move between tiers when evidence warrants it.

7.3 Biological governance

The biological zone requires institutional ethics review, biosafety protocols, provenance for tissue or cell sources, relevant donor permissions, staff training, humane experimental design and independent oversight. The unresolved ethical status and possible future welfare questions surrounding increasingly complex neural organoids should be treated conservatively [10]. HYPERION must not make claims about organoid consciousness or computational superiority without evidence. Its BPU module is a laboratory experiment interface, not a general-purpose replacement for digital computation.

8. Representative Workflows

8.1 Multimodal AI assistant

A user submits text, speech and camera input. The CPU authenticates the request and builds an execution graph; an ISP preprocesses camera signals, an SPU performs low-latency audio processing, an NPU or GPU computes perception embeddings, and a model-serving backend executes language inference on an LPU, GPU or other compatible accelerator. DPUs manage permitted data transport and storage paths; the security processor protects cryptographic material. Which accelerator wins depends on model compatibility, token throughput, first-token delay, concurrent sessions and transfer costs. A QPU or BPU would not be placed in this production path without a demonstrable, semantically appropriate subproblem.

8.2 Interactive 3D simulation

The engine partitions rendering, scene graph work, physics, audio and sensor processing. CPUs manage world-state updates; GPUs render and execute parallel physics; a PPU service provides a physics-specific backend when useful; FPGAs may provide deterministic hardware-in-the-loop interfaces; and the SPU handles spatial audio. Time-sensitive work is constrained by a frame deadline, meaning an impressive theoretical speedup on a remote processor is irrelevant if network latency exceeds the frame budget.

8.3 Scientific optimisation and sampling

A classical optimiser formulates a constrained scientific problem. The scheduler may evaluate a conventional CPU/GPU solver alongside a quantum-circuit approach or TSU-compatible probabilistic sampler, provided the mathematical mapping is valid. Equivalent problem instances, quality criteria and stopping rules are required. A valid result requires independent comparison with strong classical baselines and an accounting of total wall-clock time, calibration and energy rather than isolated accelerator kernel metrics.

8.4 Event-driven biological and neuromorphic research

A sensor or microelectrode-array signal is captured with timestamped metadata, transformed into spikes where warranted, and analysed using a neuromorphic processor. A biological research experiment may stimulate and record neuronal culture activity through separately governed protocols. The scheduler coordinates experiment requests and data collection but does not treat a living culture as an unconstrained remotely programmable chip. Its outcomes are experimental observations and must be interpreted statistically and biologically.

9. Experimental Evaluation Plan

A research paper about a proposed platform must define the conditions under which its architectural claims could be rejected. HYPERION therefore proposes an evaluation programme with preregistered workloads and a control configuration.

9.1 Baselines and ablations

Baseline B0 is a strong CPU-only implementation. B1 is a tuned CPU+GPU deployment with the same relevant models and datasets. B2 adds ordinary production accelerators under a static expert-defined mapping. H1 is the full capability-aware HYPERION scheduler over the same available production hardware. H2 adds selected research backends only for explicitly compatible experimental tasks. An ablation disables cost learning, then data locality, then confidence-aware fallback, to quantify which scheduler features actually contribute value.

This design separates the effect of **adding hardware** from the effect of **orchestrating hardware better**. It also prevents comparing a remotely hosted QPU against a weak laptop baseline when the relevant question is whether it improves on an appropriate contemporary classical algorithm.

9.2 Metrics

Table 2. Primary evaluation measures.

Category	Metric	Required reporting
Latency	End-to-end p50, p95 and p99 time	Includes queue, compile, transfer and validation
Throughput	Completed accepted jobs per second	Same output-quality threshold for all systems
Energy	Joules per accepted result	Clearly stated measurement boundary and idle power
Cost	Monetary cost per accepted result	Capital amortisation assumptions and operating cost
Scheduler quality	Regret versus best observed feasible assignment	Confidence intervals and task-specific optima
Robustness	Failure rate, recovery time, successful fallback rate	Fault injection and real faults separately
Accuracy	Task-defined correctness or statistical fidelity	Reference implementation or accepted gold standard
Security	Policy violation attempts and blocked transfers	Auditable enforcement evidence
Portability	Supported operation coverage and code-change effort	Backend matrix with failed/incompatible cases
Scientific validity	Reproducibility and instrument/protocol metadata completeness	QPU, TSU and BPU separately

Energy measurements must specify whether they include cooling, host electronics, data movement and idle facilities. Reporting only accelerator-core joules would make cross-paradigm comparisons misleading. Quantum and biological studies should additionally document useful experimental throughput, noise, calibration, scientific repeatability and measurement overhead. No hypothetical values should be reported as results.

9.3 Workload matrix

The recommended benchmark suite contains: transformer inference at several batch sizes and context lengths; dense training kernels; graph algorithms; mixed-precision linear algebra; ray tracing and rigid-body simulation; real-time audio/video pipelines; storage and networking stress tests; spiking-network event processing; calibrated optical matrix operations; quantum circuit families with rigorous classical comparators; TSU-compatible sampling tasks; and a small set of ethics-approved biological readout experiments. Every benchmark records available hardware, exact software version, data shape, numerical precision and transfer topology.

The main hypothesis is that H1 reduces cost or end-to-end time relative to B2 on a sufficiently diverse workload mix without violating correctness or policy constraints. The null hypothesis is that the added scheduling overhead and complexity eliminate the advantage. Research backends are evaluated as separate scientific experiments; their presence must not be used to imply improved general-purpose performance.

9.4 Statistical methodology

Measurements should be repeated across controlled runs with confidence intervals, warm-up and compilation distinguished from steady-state operation, and resource contention recorded. Workload order should be randomised where possible. Non-parametric analysis may be necessary for skewed latency distributions. Hardware-specific precision and quality constraints must be fixed *before* cross-device performance comparison. Report unsuccessful runs, unsupported operators and device unavailability as first-class findings rather than discarding them.

10. Deployment Strategy and Engineering Roadmap

Phase 0—Reference simulation. Build typed device descriptors, trace-driven device simulators, a DAG scheduler, mock QPU/TSU/BPU endpoints, cost models and a reproducible evaluation harness. Use CPU/GPU execution for reference correctness. This phase is achievable without acquiring experimental devices and permits transparent testing of scheduling decisions.

Phase 1—Conventional heterogeneous prototype. Add available CPUs, GPUs, NPUs, FPGAs and DPU-backed infrastructure. Integrate off-the-shelf scheduling, observability, storage and identity components. Verify real telemetry, policy and failure recovery before optimising for performance.

Phase 2—Specialised AI services. Add cloud or installed TPU, LPU and wafer-scale execution services where accessible. Implement model format conversion, compilation caching, data residency constraints and reproducible cost comparisons.

Phase 3—Non-classical research adapters. Integrate neuromorphic and photonic devices and TSU research interfaces as approved external services. Provide explicit sample and event types, calibration metadata, experiment IDs and statistical validation. Vendor roadmaps are not treated as guaranteed delivery dates.

Phase 4—Quantum and biological laboratory federation. Connect quantum endpoints through supported quantum SDKs and biological endpoints only under independent scientific and ethical governance. Maintain asynchronous job semantics, approval gates, detailed logs and separate failure domains.

At every phase, a fully functional system is defined by its verified subset of backends. “Twenty-class support” should never mean that twenty devices were automatically installed, electrically connected or benchmarked.

11. Limitations, Risks and Threats to Validity

First, the taxonomy is descriptive rather than mutually exclusive. Counting WSE, ASIC and TPU as separate categories can exaggerate architectural novelty unless their actual service and software differences are acknowledged. Second, performance is application- and implementation-specific: no accelerator guarantees a gain after format conversion, data motion or queueing. Third, vendor peak performance figures often use different precisions, sparsity assumptions and computational definitions, so they cannot be summed into a scientifically meaningful “total HYPERION FLOPS” number.

Fourth, certain backends are not generally programmable. Quantum circuits, stochastic samplers and biological systems do not execute arbitrary ordinary programs. Fifth, access limitations and capital costs may dominate deployment feasibility. Sixth, scientific governance changes the set of permissible biological experiments and may prohibit some envisioned applications. Seventh, cost-model errors, hardware drift, vendor-specific software and research-device instability can bias scheduling results. Finally, a single universal programming abstraction risks hiding essential physical properties, so HYPERION uses a shared *workflow* interface rather than an asserted universal execution semantics.

A successful study could conclude that only a subset of these processor classes deserves integration for a given application. That would be a useful finding rather than a failure of the research programme.

12. Discussion: What Would Constitute Novelty?

Heterogeneous computers, accelerator orchestration and hybrid quantum-classical workflows already exist. HYPERION’s potential research contribution would therefore lie in the **explicitly typed federation across fundamentally different computing models**, consistent cross-domain provenance, a policy-aware scheduler, and a rigorous comparison framework that includes all operational costs. Merely drawing twenty named processing units in one architecture diagram is not a technical breakthrough.

The strongest initial publication path is consequently a software systems demonstration: an implemented workload intermediate representation, prototype adapters, capability-aware scheduling and experimentally verified comparisons with strong conventional baselines. A second paper could address heterogeneous sampling semantics for TSU/QPU/neuromorphic outputs. Biological computing should be treated as a distinct collaborative research programme requiring specialist co-authorship and governance.

13. Conclusion

HYPERION-20 presents a reference architecture for combining twenty functional processor categories under a single capability-aware workflow platform. Its design rejects three unsound assumptions: that all processors are equivalent, that all must share coherent memory, and that including more computing paradigms necessarily makes a computer faster. Instead, it proposes typed execution contracts, explicit data boundaries, measured cost prediction, policy-based admission, robust fallback, and an independently testable performance methodology.

Commercial CPUs, GPUs, tensor engines, LPUs, DPUs and wafer-scale systems can provide an engineering foundation; photonic, neuromorphic, thermodynamic, quantum and biological interfaces contribute specialised research opportunities where algorithm compatibility and experimental evidence justify their inclusion. The principal expected outcome is not a single machine with a universal superiority claim, but a practical methodology for deciding *which kind of computation belongs on which kind of hardware, under what conditions, and with what evidence*. As of 1 November 2026, HYPERION-20 remains an unimplemented research proposal. The next defensible milestone is a software prototype and preregistered benchmark study.

References

[1] Google Cloud. “TPU architecture.” Official technical documentation. https://docs.cloud.google.com/tpu/docs/system-architecture-tpu-vm (accessed 1 November 2026).

[2] Groq. “The Groq LPU Explained.” Technical article, 2025. https://groq.com/blog/the-groq-lpu-explained (accessed 1 November 2026).

[3] Cerebras Systems. “Cerebras Systems Unveils World’s Fastest AI Chip with Whopping 4 Trillion Transistors.” Press release, March 2024. https://www.cerebras.ai/press-release/cerebras-announces-third-generation-wafer-scale-engine (accessed 1 November 2026).

[4] NVIDIA. “DOCA Software Framework.” Developer documentation. https://developer.nvidia.com/networking/doca (accessed 1 November 2026).

[5] Intel. “Intel Builds World’s Largest Neuromorphic System to Enable More Sustainable AI.” Newsroom, 17 April 2024. https://newsroom.intel.com/artificial-intelligence/intel-builds-worlds-largest-neuromorphic-system-to-enable-more-sustainable-ai (accessed 1 November 2026).

[6] Ahmed, S. R., et al. “Universal photonic artificial intelligence acceleration.” *Nature* 640, 368–374 (2025). https://doi.org/10.1038/s41586-025-08854-x.

[7] IBM. “IBM Quantum Computing: Hardware and roadmap.” Official technical overview. https://www.ibm.com/quantum/hardware (accessed 1 November 2026).

[8] Extropic. “Inside X0 and XTR-0.” Technical article, 29 October 2025. https://extropic.ai/writing/inside-x0-and-xtr-0 (accessed 1 November 2026); see also “Thermodynamic Computing: From Zero to One,” https://extropic.ai/writing/thermodynamic-computing-from-zero-to-one.

[9] Jordan, F. D., Kutter, M., Comby, J.-M., Brozzi, F. and Kurtys, E. “Open and remotely accessible Neuroplatform for research in wetware computing.” *Frontiers in Artificial Intelligence* 7, 1376042 (2024). https://doi.org/10.3389/frai.2024.1376042.

[10] Nuffield Council on Bioethics. *Neural Organoids: Ethical and Governance Considerations.* Report, 12 May 2026. https://www.nuffieldbioethics.org/publication/neural-organoids-ethical-and-governance-considerations/.

[11] Kubernetes. “Device Plugins.” Official documentation. https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/ (accessed 1 November 2026).

[12] OpenXLA. “PJRT Plugin Integration.” Technical documentation. https://github.com/openxla/xla/blob/main/docs/pjrt/pjrt_integration.md (accessed 1 November 2026).

[13] Linux Kernel Documentation. “Compute Express Link.” Official technical reference. https://www.kernel.org/doc/html/latest/driver-api/cxl/ (accessed 1 November 2026).

Appendix A. Proposed Workflow API Contract

A portable HYPERION task should expose the fields operation_id, operation_semantics, input_schema, output_schema, precision_or_uncertainty, allowed_device_classes, deadline, privacy_policy, failure_policy, provenance_required and validation_method. Research-only extensions include protocol_approval, calibration_record, instrument_state, sampling_plan and scientific_metadata.

An appropriate API makes unsupported computation *explicit*. For example, if an LPU backend cannot implement a requested model operator or if a TSU cannot represent the requested probability distribution, the job must fail compatibility checks or invoke an approved conversion—not silently execute a different mathematical task.

Appendix B. Proposed Reproducibility Checklist

1. Identify the exact device, firmware, backend software, compiler and driver versions.
2. State whether each result is measured, simulated, modelled, vendor-reported or proposed.
3. Record data format, numerical precision, full model configuration, workload size and seeds where relevant.
4. Include compilation, queueing, network transfers, calibration, retries and validation in end-to-end timing.
5. Report measurement boundaries for energy and financial cost.
6. Run strong reference implementations on comparable problem definitions and output-quality constraints.
7. Make device incompatibilities, failures and unavailable research systems visible in published results.
8. Record applicable ethical review and scientific protocol requirements before biological experiments.
9. Disclose any simulation assumptions and perform sensitivity analyses on scheduling predictions.
10. Publish code, synthetic workload traces, metadata schemas and non-sensitive results where permitted.